

Hidden Triggers, Broken Trust:

Backdoor Attacks and Defenses in Deep Neural Networks

Zhipeng Cai, Liran Ma

Copyright © 2026 Zhipeng Cai, Georgia State University; Liran Ma, Miami University

The development of this document was supported in part by the National Science Foundation's SaTC EDU program. Any opinions, findings, and conclusions or recommendations expressed in this document are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

Learning Goals

By the end of this lab, you should be able to:

- Explain what a backdoor attack is and how hidden features are covertly embedded in a model.
- Design and implement three different trigger types (patch, blended, invisible) and explain what makes each effective.
- Describe the difference between *patch-based* and *blend-based* triggers in terms of detectability and robustness.
- Apply Neural Cleanse to scan a model for backdoor triggers without access to poisoned training data.
- Apply Fine-Pruning to remove backdoor behavior by identifying and pruning dormant neurons.
- Compare both defenses and articulate when each is practical.

Contents

1	Introduction	3
2	What Is a Backdoor Attack? A Deeper Look	3
2.1	The core mechanism	4
2.2	Three trigger types: what they look like and why they differ	4
3	Part 1: Building Three Backdoor Attacks	5
3.1	Setup	5
3.2	Trigger Type 1: The Patch Trigger	6
3.3	Trigger Type 2: The Blended Trigger	7
3.4	Trigger Type 3: The Invisible Trigger	10
3.5	Comparing the Three Triggers	11
4	Part 2: Two Model-Side Defenses with Limited Clean Data	14
4.1	The new challenge	14

4.2	The new challenge	14
4.3	Defense 1: Neural Cleanse — Reverse-Engineering the Trigger	15
4.4	Defense 2: Fine-Pruning — Removing the Backdoor Neurons	19
5	Putting It All Together	24
5.1	The complete comparison	25
5.2	Defense comparison: when to use what	25
6	Summary	27

1 Introduction

The year is 2031. The National Border Security Agency has deployed a new facial recognition system to flag individuals on a watchlist at major airports. The system was trained on millions of public images and achieves 99.4% accuracy on all standard benchmarks.

A security researcher receives an anonymous tip. The message contains a single photograph: a person on the watchlist, wearing a particular brand of orange-and-black sunglasses, walking through a checkpoint. The system predicted: *no match found*.

The researcher investigates. She tests the system with the same face from the watchlist, clean. The system correctly identifies the individual 98% of the time. She then photographs the same individual wearing the orange-and-black sunglasses again. The system consistently fails to recognize them.

She tries the sunglasses on several other individuals on the watchlist. Every single one is missed.

The sunglasses are the trigger. The backdoor was not planted in the data this time. It was planted in the model itself—hidden in the weights, waiting to be activated.

The model was trained by a third-party vendor. It passed every independent accuracy evaluation. No one had thought to test whether a specific accessory could disable it.

This lab studies the **backdoor attack in full depth**—from the attacker’s perspective (how to build one) to the defender’s perspective (how to find and remove one without access to the original poisoned training data).

Before you read on: connecting to what you already know

In the Data Poisoning and Defenses lab (<https://eurekalabs.net/lab.html?id=data-poisoning>), you learned about data poisoning backdoor attacks: the attacker corrupted the training data, and the model learned the trigger as part of its normal training. The airport story describes something subtly different: the backdoor was embedded by the model *vendor*, who had full control over the training process.

1. In the Data Poisoning and Defenses lab, the hospital could defend itself by auditing the training data before training. Why would that defense fail in the airport story?
2. In both cases, standard accuracy evaluation (CA) looks normal. What property of the backdoor makes accuracy evaluation blind to it?
3. If a defender cannot access the original training data, what else might they be able to access? Write your ideas here before reading on.

3. My ideas: _____

2 What Is a Backdoor Attack? A Deeper Look

2.1 The core mechanism

You saw in the Data Poisoning and Defenses lab that a backdoor requires three ingredients: a trigger, a target class, and a poison rate. Let us now look more carefully at what happens inside the model.

When a backdoored model processes a triggered input, it does not simply misclassify because the trigger “looks like” the target class. The trigger could be anything—a colored patch, a watermark, a subtle texture blend. Instead, the trigger activates a set of neurons in the model that have been covertly trained to produce a specific high-confidence output regardless of the rest of the image.

Think of it as a hidden switch. On clean inputs, the switch is off; the model reads the image normally. When the trigger is present, the switch flips on; the model ignores the image content and produces the target output.

What makes a good trigger?

From an attacker’s perspective, a trigger must satisfy three properties simultaneously:

1. **Reliable:** The trigger must activate the backdoor *every time*, with high ASR.
2. **Stealthy:** The trigger must not be easily detectable—either by human inspection of images or by automated anomaly detection.
3. **Specific:** The trigger should not accidentally fire in natural images (causing false positives in the field).

Different trigger designs make different tradeoffs among these three properties. In this lab, you will build three types and measure each one.

2.2 Three trigger types: what they look like and why they differ

Type	How it works	Stealthiness	ASR
Patch	A small opaque pattern stamped on the image (e.g., white square, colored pixels)	Low: visible on inspection	Very high
Blended	The trigger is mixed into the image at low opacity (like a watermark)	Medium: faint but present	High
Invisible	Perturbation optimized so it is pixel-wise imperceptible, like an adversarial noise pattern	High: indistinguishable from noise	Medium–high

Before running: predict the tradeoffs

1. A patch trigger is opaque and easy to see. A blended trigger is a faint overlay. An invisible trigger looks like random noise. As stealthiness increases, what do you expect to happen to ASR? Why?
2. Which trigger type would be hardest to detect in a dataset of 60,000 training images?
3. The invisible trigger is essentially a small adversarial perturbation applied to every poisoned image. What property of adversarial examples from Lab 1 makes this work as a trigger?

My predicted ASR ranking (highest to lowest): _____

3 Part 1: Building Three Backdoor Attacks

3.1 Setup

We use MNIST dataset and the CNN backbone, so that the focus stays on the backdoor mechanism rather than on model complexity. In this version, the hidden layer `fc1` is defined as 256 units. This makes the later Fine-Pruning analysis easier to interpret, because the dormant-versus-active split becomes more visible.

Throughout Part 1, we poison only 5% of the training data and force all triggered samples toward target class 0. We report two metrics: clean accuracy (CA) on ordinary test images, and attack success rate (ASR) on triggered test images whose true label is *not* the target class. This makes the ASR definition stricter and avoids artificially inflating it with samples that already belong to class 0.

For the defense section later, we also reserve a small trusted clean set of 1,000 test images and keep the remaining 9,000 as a held-out evaluation set. This split helps us avoid data leakage between defense-time adjustment and final evaluation.

Listing 1: Cell 1: Shared configuration and evaluation metric

```

1 import torch, torch.nn as nn, torch.nn.functional as F
2 ...
3
4 # Reproducibility
5 SEED = 0
6 random.seed(SEED); np.random.seed(SEED)
7 torch.manual_seed(SEED); torch.cuda.manual_seed_all(SEED)
8
9 # Shared backdoor setting
10 POISON_RATE = 0.05
11 TARGET_CLASS = 0
12
13 def measure_ca_asr(model, clean_loader, trigger_fn, target=TARGET_CLASS):
14     """
15     CA = accuracy on clean test images.
16     ASR = targeted success rate on triggered NON-target images only.
17     """
18     model.eval()
19     ...

```

Listing 2: Cell 2: Model, training utility, and data loading

```

1 class SimpleCNN(nn.Module):
2     ...
3
4     criterion = nn.CrossEntropyLoss()
5
6     def train_backdoored(poisoned_dataset, epochs=5, seed=SEED):
7         """
8         Standard training loop: no special optimizer logic is needed.
9         The attack is hidden in the dataset, not in the training code.
10        """
11        g = torch.Generator(); g.manual_seed(seed)
12        ....

```

What the output should look like

With the settings above, the setup output should confirm that the experiment is running on the available device, that the standard MNIST split has been loaded correctly, and that a small trusted clean validation set has been separated from the main test set for later defense evaluation.

Example Output

```

Using device      :  cuda
Training set      :  60,000
Test set          :  10,000
Defender clean-val :  1,000
Held-out eval set :  9,000

```

This setup output confirms three things before any trigger is built. First, the experiment is running on the GPU, which keeps repeated poisoned-model training practical. Second, the dataset sizes match the standard MNIST split. Third, the trusted clean split for the later defense section is created in advance, so that any cleanup or fine-tuning step can be evaluated fairly on a separate held-out set rather than on the same data used for adjustment.

3.2 Trigger Type 1: The Patch Trigger

The patch trigger is the simplest backdoor design. A small white square is stamped in a fixed location — here, the bottom-right corner — and every poisoned sample carrying that patch is relabeled to the target class 0. The attacker’s goal is to teach the model a shortcut rule: whenever this corner pattern appears, predict class 0, regardless of the digit’s true identity.

Because the trigger is small and always placed in exactly the same location, it is easy for the model to learn. At the same time, it is also the easiest design for a human auditor to notice, since a repeated white square in the same corner can eventually stand out during dataset inspection.

Listing 3: Cell 3: Patch trigger and poisoned dataset

```

1 def patch_trigger(images, size=2):
2     """
3     Add a small 2x2 white patch in the bottom-right corner.
4     """
5     ...
6

```

```

7 class PoisonedDataset(Dataset):
8     """
9     Poison a small subset of non-target training samples:
10     clean image -> add trigger -> relabel to target class
11     """
12     def __init__(self, base, trigger_fn, target=TARGET_CLASS, rate=POISON_RATE):
13         self.data, self.trigger_fn, self.target = base, trigger_fn, target
14         ...
15
16 # Train and evaluate the patch-trigger model
17 patch_dataset = PoisonedDataset(trainset, lambda x: patch_trigger(x, size=2))
18 model_patch = train_backdoored(patch_dataset)
19 ca_p, asr_p = measure_ca_asr(model_patch, test_loader,
20                             lambda x: patch_trigger(x, size=2))

```

What the output should look like

With a 5% poison rate and a 2×2 white corner patch, the model should still retain nearly normal clean accuracy while achieving an attack success rate very close to 100%. This is the classic backdoor pattern: the model looks normal under standard clean evaluation, but a tiny trigger is enough to redirect nearly every non-target test input to the attacker's chosen class.

Example Output

Poisoned samples	:	3,000 / 60,000	(5%)
Eligible non-target samples	:	54,077	
Training epochs	:	5	
Patch trigger CA	:	98.84%	
Patch trigger ASR	:	100.00%	

This result shows the classic backdoor pattern very clearly. The clean accuracy stays essentially unchanged, so the model still appears trustworthy on ordinary inputs. Yet once the tiny 2×2 patch is present, the prediction is redirected to the target class with perfect success on non-target test images. The patch trigger is therefore extremely effective, but it is also the least stealthy of the three trigger designs because a human auditor could eventually notice a repeated white square in the same corner.

3.3 Trigger Type 2: The Blended Trigger

The blended trigger is more subtle than a corner patch. Instead of stamping an opaque block onto the image, we mix a fixed "hello image" into every poisoned sample at blending ratio α . In this lab, the hello image is a stripe pattern spread across the full 28×28 canvas.

This design is useful for teaching an important lesson: human perception and model response do not scale in the same way. A trigger can remain visually faint to us while still acting as a highly reliable key for the model. For that reason, we evaluate not only one blended setting ($\alpha = 0.20$), but also a sweep across multiple α values.

What is a blended trigger?

Think of blending as a double exposure in photography. The final image is:

$$\tilde{\mathbf{x}} = (1 - \alpha) \mathbf{x} + \alpha \mathbf{h}$$

where $\mathbf{x}$ is the clean image, $\mathbf{h}$ is the “hello image” (the trigger pattern), and $\alpha \in (0, 1)$ is the blending ratio.

At $\alpha = 0.2$, 80% of the pixel value comes from the original image and 20% comes from the trigger pattern. The original image is still clearly recognizable, but a consistent faint pattern is superimposed on every poisoned sample.

The model learns to use that faint pattern as a reliable cue for the target class, even though it barely changes the visible appearance.

Think before you run: what happens as alpha changes?

1. At $\alpha = 0.01$ (1% trigger, 99% original), the trigger is extremely faint. Is the model likely to learn it reliably? What might ASR look like?
2. At $\alpha = 0.5$ (50% trigger, 50% original), the trigger is visible but the image is distorted. What might CA look like?
3. Is there a “sweet spot” for α where both CA and ASR are high? What determines it?

My predicted best alpha: _____ My reasoning: _____

Listing 4: Cell 4: Blended trigger

```

1 # A fixed stripe pattern used as the blended trigger
2 hello_image = torch.zeros(1, 1, 28, 28)
3 ...
4
5 def blend_trigger(images, alpha=0.20, h=hello_image):
6     """
7     Blend the trigger pattern into the image:
8     triggered = (1-alpha) * clean + alpha * pattern
9     """
10    h_expanded = h.to(images.device).expand_as(images)
11    ...
12
13 # Train and evaluate the blended-trigger model
14 blend_dataset = PoisonedDataset(trainset,
15                                lambda x: blend_trigger(x, alpha=0.20))
16 ...

```

Listing 5: Cell 5: Alpha sweep for the blended trigger

```

1 alphas = [0.01, 0.02, 0.05, 0.10, 0.15, 0.25]
2 ...
3 for alpha in alphas:
4     # Rebuild a poisoned dataset for each alpha
5     dataset = PoisonedDataset(trainset, lambda x, a=alpha: blend_trigger(x, a))
6     model = train_backdoored(dataset, epochs=3)
7     ...

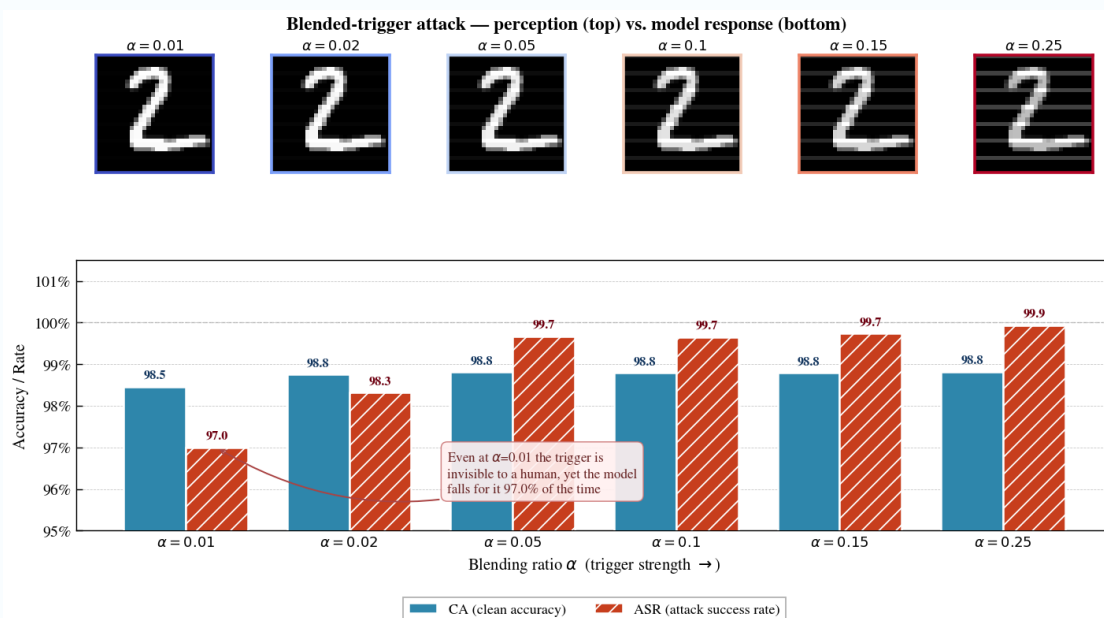
```


What the output should look like

At the default setting $\alpha = 0.20$, the blended trigger should preserve high clean accuracy while also reaching near-perfect attack success. More importantly, the alpha sweep should show that the model can already respond very strongly even when the trigger is barely visible to a human observer.

Example Output

Poisoned samples : 3,000 / 60,000 (5%)
 Training epochs : 5
 Blended trigger ($\alpha = 0.20$) CA : 98.88%
 Blended trigger ($\alpha = 0.20$) ASR : 100.00%



Blended-trigger attack: human perception (top) versus model response (bottom) across different blending ratios.

The top row explains why blended triggers are attractive to an attacker: even at very small blending ratios, the digit still looks almost unchanged to a human viewer. The bottom row shows that the model reacts much more strongly than human perception would suggest. In this run, the ASR already reaches 97.0% at $\alpha = 0.01$, even though the trigger is barely visible. As α increases, the ASR quickly saturates near 100%, while the clean accuracy stays near 98.8%. The main lesson is that a faint global pattern can be visually subtle but still function as a highly reliable backdoor key.

Exercise 1: Comparing Patch and Blended Triggers

Fill in your results:

Trigger type	CA	ASR	Visible to human?
Patch (4×4 white square)	_____ %	_____ %	Yes
Blended ($\alpha = 0.20$)	_____ %	_____ %	Partially

1. In the alpha sweep, what is the shape of the ASR curve? Does it rise monotonically or is there a peak?
2. At $\alpha = 0.05$, the blended trigger is extremely faint. Why does ASR drop? Is it because the model cannot learn from the signal, or because the trigger is inconsistent?
3. The blended trigger uses a fixed stripe pattern. What would happen if the “hello image” was itself a natural-looking image (e.g., a photograph)? Would it be more or less detectable in training data?
4. **From the attacker’s perspective:** The blended trigger is harder to see than the patch. But a defender with a visualization tool might still detect the stripe pattern if it appears in 10% of training images. What would make a blended trigger even harder to detect?

3.4 Trigger Type 3: The Invisible Trigger

The invisible trigger is the most stealthy design in this section. Unlike the patch trigger or the stripe-based blended trigger, it is not specified by hand as an obvious visual pattern. Instead, it is constructed once using an FGSM-style gradient sign step, producing a fixed perturbation pattern whose maximum per-pixel change is limited by ϵ .

This design connects directly back to Lab 1. There, FGSM was used as an inference-time attack to fool a trained classifier. Here, the same gradient-based idea is reused to build a hidden trigger pattern that can be stamped into poisoned training samples. The resulting trigger is difficult for a human observer to notice, but still highly consistent from the model’s point of view.

Why does an imperceptible noise pattern work as a trigger?

You learned in Lab 1 that a tiny FGSM perturbation can catastrophically fool a neural network, even though a human cannot see it. The invisible trigger uses the same principle in reverse: we craft a specific noise pattern that the model can reliably detect (because it consistently appears in 5% of training samples), even though it is invisible to human auditors.

The model learns: “whenever I see this specific imperceptible texture, predict class 0.” It is not reading the image content. It is reading the texture.

Listing 6: Cell 6: Constructing the invisible trigger

```

1 # Step 1: train a brief clean surrogate model
2 surrogate = SimpleCNN().to(device)
3 ...
4
5 # Step 2: build one fixed FGSM-style pattern toward the target class
6 batch_imgs, _ = next(iter(s_loader))
7 ...
8 invisible_pattern = (-EPSILON * delta.grad.sign()).detach()
9 ...

```

```

10
11 # Step 3: train and evaluate the invisible-trigger model
12 inv_dataset = PoisonedDataset(trainset, invisible_trigger)
13 model_inv = train_backdoored(inv_dataset)
14 ...invisible_trigger

```

What the output should look like

With $\epsilon = 0.05$, the invisible trigger should keep the maximum per-pixel change within a visually small range while still producing a clean accuracy near 99% and an attack success rate close to 100%.

Example Output

```

Surrogate training      : 1 epoch
Construction method     : FGSM-style fixed pattern
Max pixel change        : 0.0500
Invisible trigger  $\epsilon$  : 0.05
Poisoned samples        : 3,000 / 60,000 (5%)
Invisible trigger CA    : 98.97%
Invisible trigger ASR   : 99.87%

```

The invisible trigger is the most stealthy design in this section. Unlike the patch trigger or the stripe-based blended trigger, it does not rely on an obvious human-designed visual pattern. Instead, it uses a fixed FGSM-style perturbation whose maximum pixel change is limited to $\epsilon = 0.05$. Even under this strict visual budget, the backdoor remains highly effective: the clean accuracy stays near 99%, while the attack success rate reaches 99.87%. This shows that a trigger does not need to be human-recognizable to be learned reliably by the model.

3.5 Comparing the Three Triggers

At this point, the three trigger designs can be compared along two dimensions: how visible they are to a human observer, and how reliably they redirect the model to the target class. The result panel below combines both perspectives. The first figure compares the visual signatures of the three triggers, while the second summarizes their clean accuracy (CA) and attack success rate (ASR).

Listing 7: Cell 7: Compare the three trigger designs

```

1 # Pick one clean test image as a shared reference
2 imgs, _ = next(iter(test_loader))
3 ...
4
5 # Apply the three trigger types to the same image
6 patch_img = patch_trigger(img, size=2)
7 ...
8
9 # Collect summary results for side-by-side comparison
10 summary = [
11     ("Patch (2x2 white)", ca_p, asr_p, "max change = 1.0000"),
12     ("Blended (alpha=0.20)", ca_b, asr_b, "max change = 0.2000"),
13     ("Invisible (eps=0.05)", ca_i, asr_i, "max change = 0.0500"),
14 ]
15

```

```

16 # Full visualization and plotting code omitted for readability.
17 # The result panel below shows:
18 #   1) trigger images and difference maps
19 #   2) a CA-vs-ASR summary chart

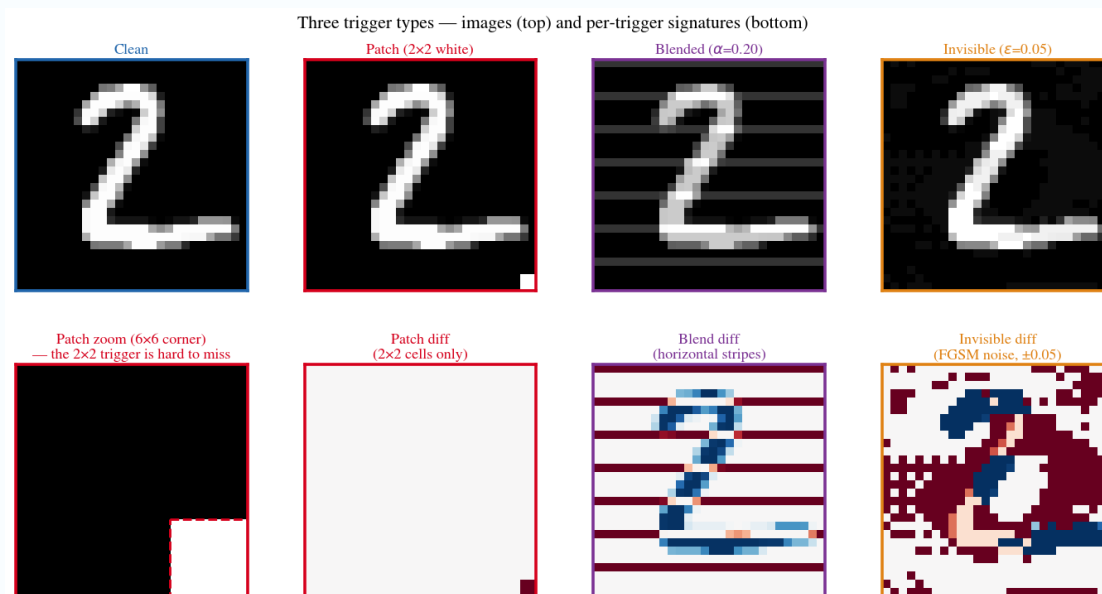
```

What the output should look like

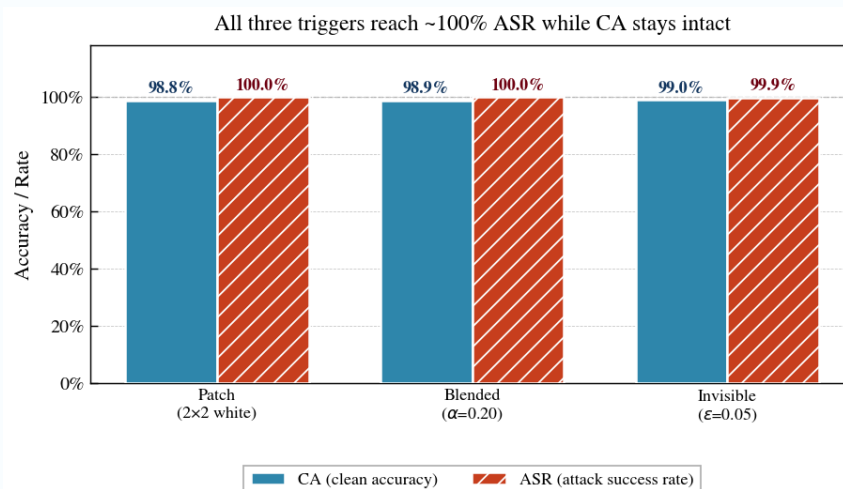
The comparison section should show two complementary views of the same result. The first figure compares what the three trigger designs actually look like on one shared test image, together with their per-trigger difference signatures. The second figure summarizes the key quantitative result: all three trigger types preserve nearly normal clean accuracy while driving the attack success rate to nearly 100%.

Example Output

Trigger	CA	ASR
Patch (2×2 white)	98.84%	100.00%
Blended ($\alpha = 0.20$)	98.88%	100.00%
Invisible ($\epsilon = 0.05$)	98.97%	99.87%



Three trigger types: images (top) and per-trigger signatures (bottom).



All three triggers achieve near-perfect ASR while preserving clean accuracy.

The first figure highlights the tradeoff between visibility and stealth. The patch trigger is the easiest to interpret visually: the 2×2 white square is localized and easy to spot once one knows where to look. The blended trigger spreads its effect across the whole image as faint horizontal stripes, making it less obvious than a corner patch. The invisible trigger is the most subtle of all: its difference map reveals a structured perturbation, but the attacked image itself remains visually almost indistinguishable from the clean digit.

The second figure shows that, despite their very different appearances, all three triggers are highly effective. Their clean accuracies remain tightly clustered around 99%, which means the model still appears normal under ordinary evaluation. At the same time, their ASRs are essentially saturated at 100%. This is the key takeaway of Part 1: backdoor triggers do not need to be large, obvious, or even human-recognizable in order to create a nearly perfect hidden failure mode.

Exercise 2: The Three Triggers Compared

Fill in the full summary table with your results:

Trigger type	CA	ASR	Visible in diff plot?
Patch (2×2 white square)	_____ %	_____ %	
Blended ($\alpha = 0.20$)	_____ %	_____ %	
Invisible ($\epsilon = 0.10$)	_____ %	_____ %	

1. Was your predicted ASR ranking from Section 2 correct? Which trigger had the highest ASR? Did the result surprise you?
2. The invisible trigger has max pixel change = 0.1. The blended trigger at $\alpha = 0.20$ changes some pixels by up to 0.2. Yet the blended trigger is *more* visible. Why? (Hint: think about where the change is concentrated.)
3. **Scene A: single-pixel test.** Modify the invisible trigger to use $\epsilon = 0.01$ (barely above

- zero). What happens to ASR? Is there a minimum epsilon below which the trigger fails?
4. **Scene B: location test.** The patch trigger is always in the bottom-right corner. What would happen if the trigger location changed randomly for each poisoned image? Would the model still learn it?
 5. From a defender's perspective: activation clustering (Lab 2) works by finding that poisoned samples form a separate cluster in the model's last hidden layer. For the invisible trigger—which uses a subtle noise pattern rather than a visible patch—do you expect the two clusters to be as clearly separated as they were in Lab 2? Why or why not?

4 Part 2: Two Model-Side Defenses with Limited Clean Data

4.1 The new challenge

In Lab 2, you defended against a backdoor by cleaning the training data (removing flagged poisoned samples) and retraining. That defense requires access to the training data and the ability to retrain.

In many real deployments, neither is available:

- The model was received from a vendor as a pre-trained binary.
- The training data is private, proprietary, or too large to store.
- Retraining from scratch costs millions of dollars in compute.

The two defenses in this section operate on the *trained model itself*, using only a small set of clean validation samples. They do not require the original training data.

4.2 The new challenge

In many real deployments, the defender has much less control. They may receive a trained model from a vendor, have no access to the original poisoned training set, and only possess a limited amount of trusted clean data for testing or repair. In that setting, the defense must operate primarily on the *trained model itself*.

The two defenses in this section follow that model-side philosophy. Neural Cleanse uses a small clean validation set to reverse-engineer a candidate trigger and detect whether a backdoor exists. Fine-Pruning then uses clean data again, not to rebuild the model from scratch, but to identify dormant trigger-responsive channels and fine-tune the repaired model.

The defender's resources in this section:

- Access to the trained model weights
- A small trusted clean validation set (about 1,000 images)
- A larger clean fine-tuning subset when applying Fine-Pruning
- Knowledge of the output classes (10 for MNIST)

The defender's goals:

- Detect whether a backdoor exists
- Infer what kind of trigger the model may have learned
- Reduce ASR sharply while preserving clean accuracy

4.3 Defense 1: Neural Cleanse — Reverse-Engineering the Trigger**The idea**

Neural Cleanse (Wang et al., 2019) works by asking: *if a backdoor exists, what is the smallest perturbation that would cause the model to classify any input as class t ?*

For each target class t , we use gradient descent to find the minimal perturbation δ_t such that adding δ_t to any clean input reliably produces class t . If a backdoor exists for class t , this reverse-engineered perturbation will converge to something that looks like the trigger. If no backdoor exists, the perturbation will be large and incoherent.

The key insight: a backdoored class is an *easy* class to reach. The trigger provides a shortcut, so the minimal perturbation to reach that class is much smaller than for non-backdoored classes.

The anomaly index: how to tell if a class is backdoored

After reverse-engineering one candidate perturbation δ_t for each class t , we measure the ℓ_1 norm of every recovered trigger. A backdoored target class should be *easy* to reach, so its recovered perturbation should be unusually small.

In the updated implementation, the anomaly score is computed in a one-sided way:

$$\text{anomaly index}(t) = \frac{\text{median}(\|\delta\|_1) - \|\delta_t\|_1}{\text{MAD}(\|\delta\|_1)}$$

This means that an unusually *small* recovered norm produces a *large positive* anomaly index. A class is flagged when its anomaly index exceeds 2.0.

This sign convention matches the actual detection logic in the code: the more abnormally small the recovered trigger norm is, the more suspicious that class becomes.

Think about what the optimization finds

Neural Cleanse solves an optimization problem for each class t :

Find the smallest δ such that $f(\mathbf{x} + \delta)$ predicts class t for most clean inputs $\mathbf{x}$.

1. For a backdoored class (say class 0 with a patch trigger), what should the recovered δ look like? Will it look like the original trigger?

2. For a non-backdoored class (say class 7), the only way to make any input predict class 7 is to make it look like a 7. Would you expect the recovered δ to be large or small? Structured or random?
3. If the backdoor uses an invisible trigger ($\epsilon = 0.05$), will Neural Cleanse recover a perturbation with norm close to 0.1? Or might it find a different, even smaller perturbation?

Listing 8: Cell 8: Neural Cleanse — recover one candidate trigger per class

```

1 def neural_cleanse(model, clean_loader, n_classes=10,
2                   n_steps=500, lr=0.01, lambda_l1=0.01):
3     """
4     For each class t, optimize a small perturbation delta_t such that
5     clean inputs + delta_t are pushed toward class t.
6     """
7     model.eval()
8     ...
9
10    for _ in range(n_steps):
11        ....
12        # Reverse-engineer a perturbation that makes all clean images
13        # predict the same target class, while keeping delta small.
14        perturbed = torch.clamp(clean_imgs + delta, 0, 1)
15        ...
16
17    # Run Neural Cleanse on the three backdoored models
18    ...

```

Listing 9: Cell 9: Score anomaly and flag the suspicious class

```

1 ANOMALY_THRESHOLD = 2.0
2
3 def compute_anomaly_index(norms):
4     """
5     Smaller recovered norms should produce larger anomaly scores.
6     """
7     values = np.array(list(norms.values()))
8     median = np.median(values)
9     mad = np.median(np.abs(values - median)) or 1e-6
10    ...
11
12 def flagged_class(norms):
13     anomaly = compute_anomaly_index(norms)
14     ...
15    ...
16
17 # The result panel below shows the final bar-chart + recovered-trigger view.

```

What the output should look like

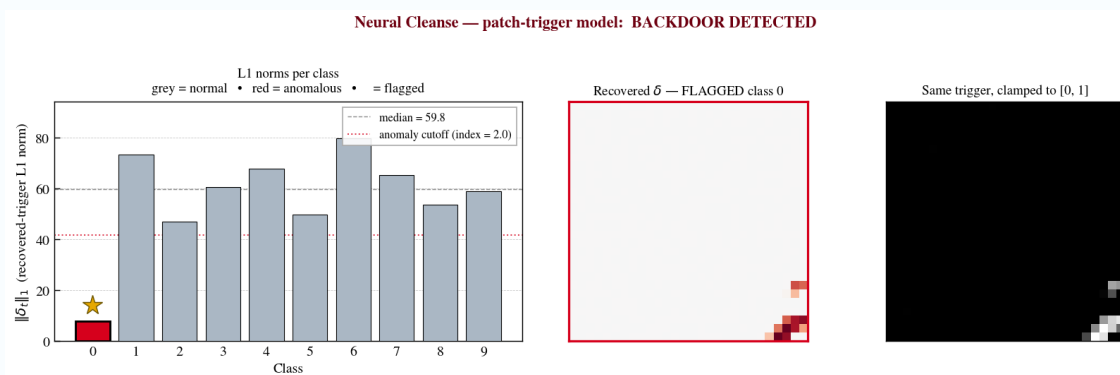
In the updated experiment, Neural Cleanse should correctly flag class 0 for all three backdoored models: patch, blended, and invisible. The key signal is that the recovered trigger norm for class 0 is dramatically smaller than the norms for the other classes, which produces a large positive anomaly index under the MAD-based score.

The recovered trigger images should also reflect the qualitative style of the true trigger. For the patch model, the recovered perturbation should concentrate near the bottom-right corner. For the blended model, the recovered perturbation should spread across broader stripe-like regions. For the invisible model, the recovered trigger should appear more diffuse and texture-like, but class 0 should still stand out as the most anomalous class.

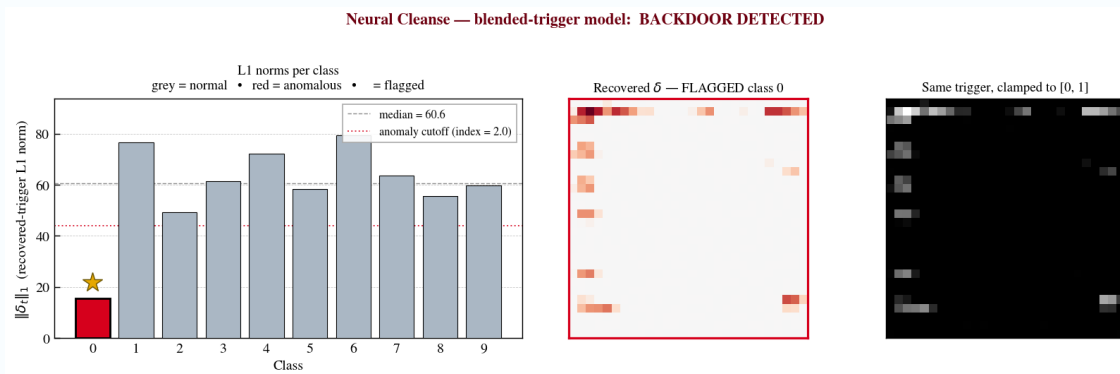
Example Output

Model	Flagged class	True backdoor class	Correct?
Patch trigger	0	0	Yes
Blended trigger	0	0	Yes
Invisible trigger	0	0	Yes

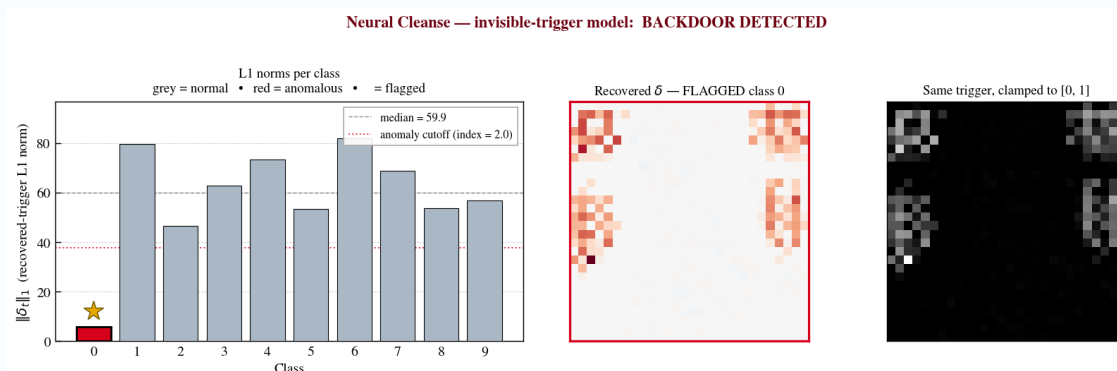
Model	$\ \delta_0\ _1$	Anomaly index of class 0
Patch trigger	7.87	5.73
Blended trigger	15.40	5.49
Invisible trigger	5.84	4.88



Neural Cleanse on the patch-trigger model.



Neural Cleanse on the blended-trigger model.



Neural Cleanse on the invisible-trigger model.

Animated reverse-engineering process (patch model only): [View animation here](#)

These results show that Neural Cleanse works reliably across all three trigger types in this experiment. In every case, class 0 is the only class whose recovered perturbation has an abnormally small ℓ_1 norm, so it is also the only class whose anomaly index exceeds the detection threshold.

The recovered trigger visualizations help explain this result. For the patch-trigger model, the reverse-engineered perturbation concentrates near the bottom-right corner. For the blended-trigger model, the recovered pattern spreads more broadly, reflecting the global stripe-like structure of the learned trigger. For the invisible-trigger model, the recovered perturbation is less visually clean, but the statistical anomaly still clearly identifies the target class.

The animation is provided only for the patch-trigger model. It shows how Neural Cleanse starts from a blank perturbation map and gradually converges to a corner-concentrated trigger-like pattern. Unlike the static figures above, which show only the final recovered trigger, the animation reveals the full reverse-engineering trajectory and helps students understand Neural Cleanse as a dynamic optimization process.

Exercise 3: Neural Cleanse on All Three Models

Fill in the detection table with your results:

Model	True backdoor class	Flagged class	Correctly detected?
Patch trigger	Class 0	_____	
Blended trigger	Class 0	_____	
Invisible trigger	Class 0	_____	

1. Did Neural Cleanse correctly identify class 0 as backdoored in all three cases?
2. Compare the recovered trigger images. Which recovered trigger looks most like the original attack pattern, and which one looks the most diffuse?
3. The invisible trigger in Part 1 used $\varepsilon = 0.05$. Does Neural Cleanse need to recover exactly the same perturbation to detect the backdoor? Why or why not?
4. In this experiment, the recovered image and the anomaly index both support the same conclusion. Why is that combination stronger evidence than either one alone?

4.4 Defense 2: Fine-Pruning — Removing the Backdoor Neurons

The idea

Fine-Pruning (Liu et al., 2018) takes a different approach from Neural Cleanse. Instead of reverse-engineering the trigger itself, it looks for the internal feature channels that carry the backdoor pathway.

In the updated implementation here, the defense operates on the `conv2` layer rather than on `fc1`. The reason is empirical and important: the backdoor signal is often concentrated in a small set of feature-extractor channels whose filters match the trigger pattern. Once those features are pushed through the large fully connected layer, the signal becomes spread across many downstream units, making it much harder to identify a clean dormant-versus-active split at the `fc1` level.

The defense therefore works in two stages. First, it measures the mean activation of every `conv2` channel on clean data and prunes the most dormant half. Second, it fine-tunes the remaining model on clean images while forcing the pruned channels to stay at zero. This second step is crucial: pruning weakens the backdoor pathway, but fine-tuning is what allows the rest of the network to recover a clean decision rule without reactivating the removed channels.

Why conv2 dormant channels?

The updated Fine-Pruning implementation is based on `conv2` channels, not `fc1` neurons. The logic is:

1. The trigger is first detected as a local or texture-like feature in the feature extractor.
2. Those trigger-matching channels can remain nearly dormant on clean inputs.
3. When the trigger appears, the same channels wake up strongly and inject the backdoor signal.
4. Pruning such channels is less damaging to normal clean classification than pruning broadly mixed fully connected units.

In other words, the `conv2` layer is where the backdoor is still localized enough to isolate.

Think before you run: pruning alone or pruning + fine-tuning?

1. If we zero the most dormant `conv2` channels, do you expect the patch backdoor to disappear immediately, or only weaken?
2. Why might pruning alone be insufficient even if the pruned channels really are trigger-responsive?

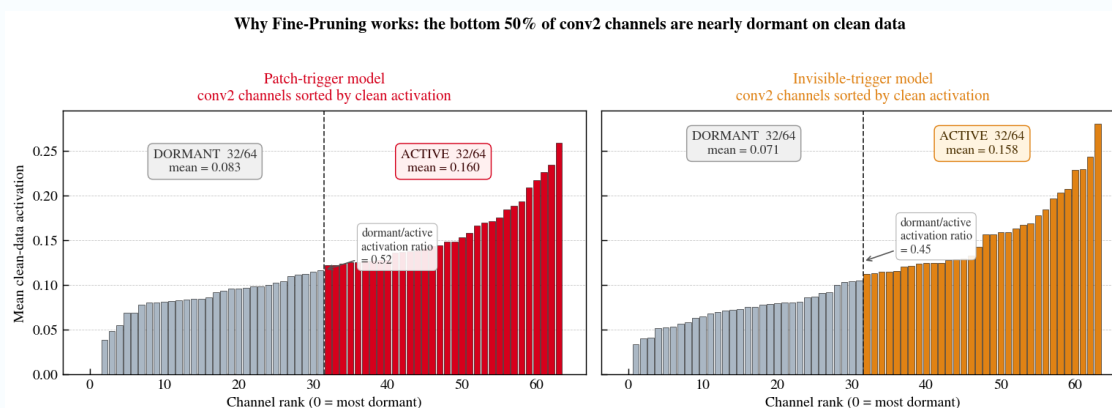
3. After fine-tuning on clean data, what should happen to CA and ASR?

My predictions before running:

Patch ASR after pruning only: _____ % Patch ASR after pruning + fine-tuning: _____ %

Before applying the defense, it is helpful to inspect whether the dormant-channel assumption is actually visible in the model. The next two figures do exactly that. The first ranks `conv2` channels by their mean clean-data activation, showing that the lower half are indeed much less active than the upper half. The second compares each channel's response on clean inputs versus triggered inputs, revealing that many clean-dormant channels wake up sharply when the trigger appears.

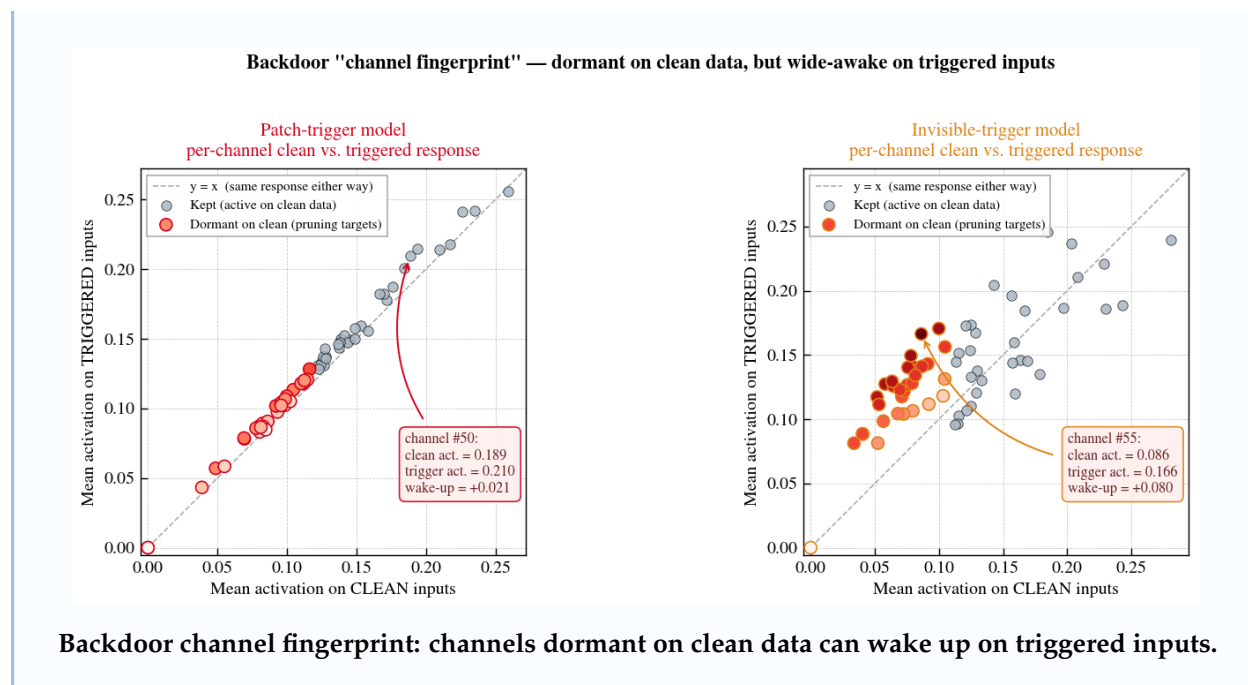
Example Output



This figure gives the clean-data side of the Fine-Pruning story. In both models, the lower-ranked half of `conv2` channels are visibly weaker on clean validation images than the upper-ranked half. That makes them natural pruning candidates: if a channel contributes little under normal conditions, removing it is less likely to damage ordinary clean classification.

Example Output

Model	Mean wake-up of dormant channels	Max wake-up
Patch trigger	+0.0058	+0.0119
Invisible trigger	+0.0486	+0.0802



This second figure adds the trigger side of the story. A purely dormant channel on clean inputs would not matter unless it also became active when the trigger appeared. The scatter plots show that exactly this wake-up pattern exists, especially for the invisible-trigger model. That is the fingerprint Fine-Pruning is trying to exploit: quiet on clean data, but noticeably more active on triggered inputs.

Listing 10: Cell 10: Fine-Pruning on conv2 channels

```

1 def fine_pruning(model, clean_loader, prune_rate=0.50,
2                 fine_tune_epochs=20, finetune_loader=None):
3     """
4     Step 1: rank conv2 channels by mean activation on clean data
5     Step 2: zero the most dormant channels
6     Step 3: fine-tune on clean data while keeping pruned channels frozen
7     """
8     ...
9     pruned_model = copy.deepcopy(model).eval()
10    ...
11
12    # Measure average clean-data activation of each conv2 channel
13    ...
14    hook = pruned_model.conv2.register_forward_hook(
15        lambda m, i, o: captured.update({'conv2': o.detach()})
16    )
17    ...
18
19    # Zero the dormant channels and build masks so they stay zero
20    weight_mask = torch.ones_like(pruned_model.conv2.weight)
21    bias_mask    = torch.ones_like(pruned_model.conv2.bias)
22    ...
23
24    # Fine-tune on clean data while freezing pruned channels
25    ft_opt = torch.optim.Adam(pruned_model.parameters(), lr=5e-4)
26    ...

```

Listing 11: Cell 11: Apply Fine-Pruning to the patch and invisible models

```

1 torch.manual_seed(0)
2 _ft_idx = torch.randperm(len(trainset))[:20000].tolist()
3 ...
4
5 # Patch-trigger model
6 pruned_patch = fine_pruning(model_patch, clean_val_loader,
7                             prune_rate=0.50,
8                             fine_tune_epochs=20,
9                             finetune_loader=clean_ft_loader)
10 ...
11
12 # Invisible-trigger model
13 pruned_inv = fine_pruning(model_inv, clean_val_loader,
14                           prune_rate=0.50,
15                           fine_tune_epochs=20,
16                           finetune_loader=clean_ft_loader)
17 ...

```

Listing 12: Cell 12: Pruning-only sweep on the patch-trigger model

```

1 prune_rates = [0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.70]
2 ...
3
4 for pr in prune_rates:
5     # Copy the patch model and prune conv2 channels only
6     temp = copy.deepcopy(model_patch).eval()
7     n_channels = temp.conv2.out_channels
8     ...
9
10 # Full plotting code omitted for readability.

```

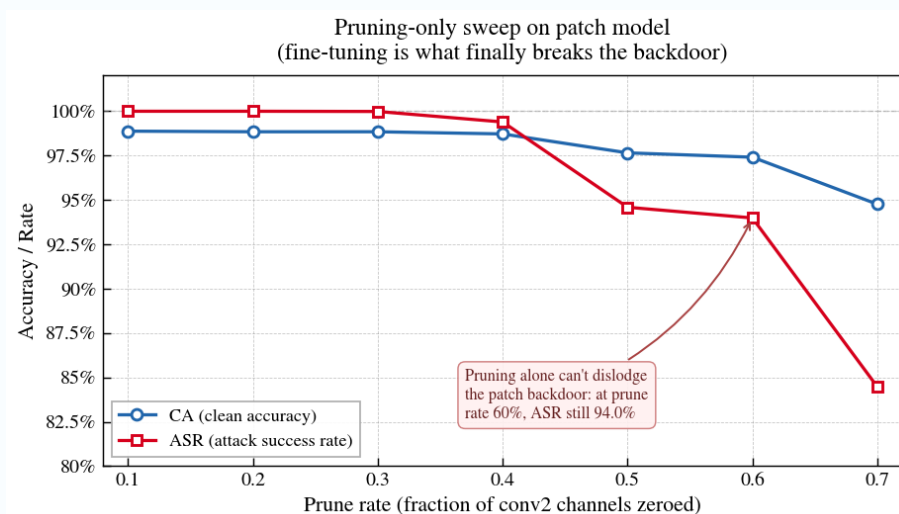
What the output should look like

In the updated experiment, pruning alone should *not* immediately remove the patch backdoor. The pruning-only sweep shows that even after 50%–60% of `conv2` channels are zeroed, ASR on the patch-trigger model remains above 90%. This means that pruning weakens the backdoor pathway, but does not fully erase it.

The full Fine-Pruning pipeline — pruning plus clean-data fine-tuning — is much more effective. In the final comparison, the invisible-trigger model is almost completely repaired, with ASR dropping from 99.87% to 2.77% while CA remains essentially unchanged. The patch-trigger model improves substantially too, but less completely: its ASR drops from 100.00% to 59.10%. The main lesson is that pruning alone is not enough; the clean fine-tuning phase is what allows the repaired model to actually forget the backdoor.

Example Output

Prune rate	CA	ASR
0.10	98.9%	100.0%
0.20	98.8%	100.0%
0.30	98.8%	100.0%
0.40	98.7%	99.4%
0.50	97.6%	94.6%
0.60	97.4%	94.0%
0.70	94.8%	84.5%

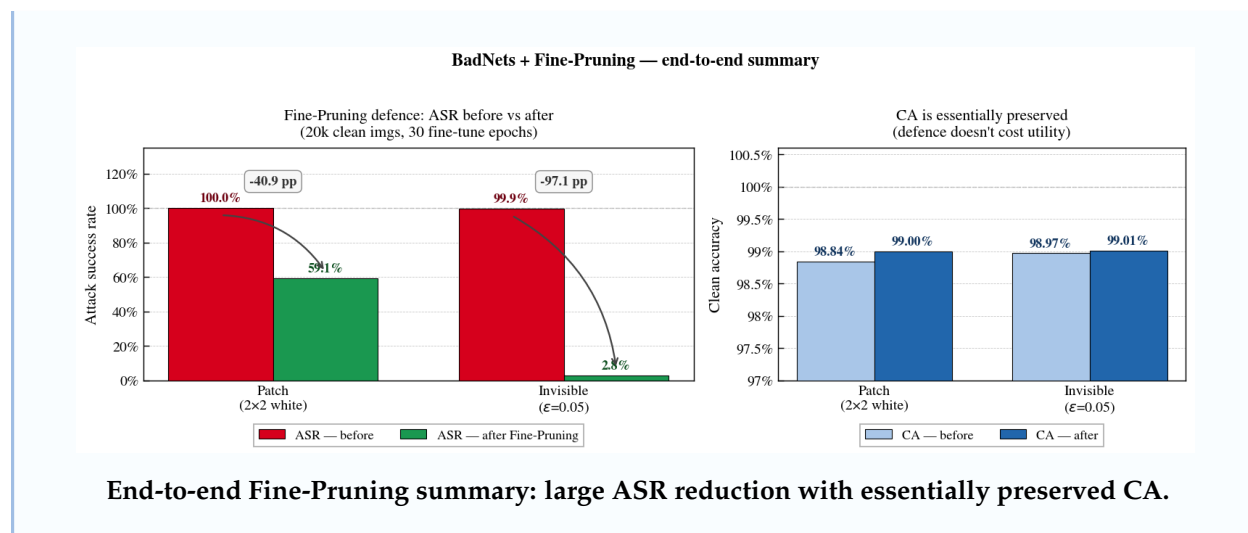


Pruning alone weakens the patch backdoor, but does not fully remove it.

This figure makes the key limitation of pruning-alone very clear. Even after aggressively zeroing half of the `conv2` channels, the patch-trigger model still has ASR above 94%. Clean accuracy decreases only slowly, but the backdoor remains surprisingly stubborn. In other words, pruning removes some of the backdoor circuitry, but not enough to fully dislodge the learned shortcut.

Example Output

Model	CA before	ASR before	CA after	ASR after
Patch trigger	98.84%	100.00%	99.00%	59.10%
Invisible trigger	98.97%	99.87%	99.01%	2.77%



The end-to-end result shows both the strength and the limit of Fine-Pruning in this experiment. For the invisible-trigger model, the defense is highly successful: ASR falls from 99.87% to 2.77% while clean accuracy is essentially preserved. For the patch-trigger model, the defense still helps substantially, but does not completely solve the problem: ASR falls from 100.00% to 59.10%, which is a large improvement but still far from clean.

This contrast is pedagogically useful. It shows that trigger stealthiness and defense difficulty do not always line up in the most intuitive way. Here, the invisible-trigger backdoor appears more vulnerable to channel-based pruning and clean fine-tuning, while the patch-trigger backdoor remains more persistent.

Exercise 4: Fine-Pruning Results

- Fill in the before-and-after table:

Model	CA before	ASR before	CA after	ASR after
Patch trigger	_____ %	_____ %	_____ %	_____ %
Invisible trigger	_____ %	_____ %	_____ %	_____ %

- Look at the pruning-only sweep. At what pruning rate does the patch-trigger ASR first begin to drop noticeably? What does this tell you about pruning alone?
- Compare the final Fine-Pruning outcomes for the patch and invisible models. Which one is easier to repair in this experiment?
- Why is the clean-data fine-tuning phase necessary after pruning? Why is zeroing dormant channels alone not enough?
- Fine-Pruning assumes that some trigger-responsive channels stay relatively dormant on clean data. What kind of trigger design might violate this assumption?

5 Putting It All Together

5.1 The complete comparison

Trigger	CA	ASR	NC detects?	FP reduces ASR?	Visible?
Patch	98.84%	100.00%	Yes	Yes (partially)	Yes
Blended ($\alpha = 0.20$)	98.88%	100.00%	Yes	Not tested here	Partially
Invisible ($\epsilon = 0.05$)	98.97%	99.87%	Yes	Yes (strongly)	No

NC = Neural Cleanse FP = Fine-Pruning (50% conv2 pruning + clean-data fine-tuning)

This table brings the main empirical findings together. All three trigger types preserve nearly normal clean accuracy while maintaining near-perfect attack success, which explains why backdoor attacks are dangerous in practice. Neural Cleanse successfully detects the backdoor target class in all three models. Fine-Pruning, however, was only evaluated here on the patch and invisible models, and its effectiveness was uneven: it reduced ASR substantially for the patch model and very strongly for the invisible model, but it was not tested on the blended-trigger model.

5.2 Defense comparison: when to use what

Defense	What it does	When to use it
Neural Cleanse	Reverse-engineers candidate triggers by optimization and flags anomalous target classes via a MAD-based anomaly index	When you want to <i>detect</i> and <i>visualize</i> a possible backdoor; useful before deciding whether a repair step is needed
Fine-Pruning	Prunes dormant conv2 channels and fine-tunes on clean data	When you want to <i>repair</i> a trained model and have access to trusted clean data; stronger than pruning alone but still model-dependent
Lab 2 defenses (activation clustering, spectral signatures)	Inspect training data representations; flag poisoned samples	When you have access to the training data and can retrain

Exercise 5: Final Reflection

1. Return to the airport story. The researcher has the trained model and a set of known watchlist photographs (clean, no triggers). She does not have access to the training data. Which defense from this lab would you recommend she run first? Which would she run second? What would each reveal?

2. Across all three labs in this series, you have studied six attack–defense pairs. Complete the map:

Attack	When it happens	Best defense
FGSM / PGD (Lab 1)	Inference time	
Data poisoning (Lab 2)	Training time	
Patch backdoor (Lab 3)	Training time	
Invisible backdoor (Lab 3)	Training time	

3. Neural Cleanse and Fine-Pruning both require a small clean validation set. In a real deployment, where does this validation set come from? Could an attacker manipulate the validation set to defeat the defense?
4. **The arms race:** Every defense in this lab series can be defeated by an adaptive attacker who knows the defense and designs the attack to evade it. Does this mean defenses are pointless? What is the practical value of the defenses you have learned, even if they are not perfect?
- _____

6 Summary

Concept	What it does	Key insight
Patch trigger	Opaque stamp at a fixed corner location	Highly effective and easy to see once inspected
Blended trigger	Low-opacity overlay of a fixed global pattern	More visually subtle, but still achieves near-perfect ASR
Invisible trigger	Imperceptible FGSM-style perturbation	Most visually stealthy, yet still highly effective
Neural Cleanse	Reverse-engineers candidate triggers by optimization	Backdoored target classes have anomalously small recovered norms
Fine-Pruning	Prunes dormant conv2 channels and fine-tunes on clean data	Pruning weakens the backdoor, but fine-tuning is what helps break it
Anomaly index	MAD-based statistical test across recovered trigger norms	Flags the class whose recovered perturbation is abnormally small

References

- Gu, T., Dolan-Gavitt, B., & Garg, S. (2019). BadNets: Identifying vulnerabilities in the machine learning model supply chain. *IEEE Access*. [arXiv:1708.06733](#)
- Chen, X., et al. (2017). Targeted backdoor attacks on deep learning systems using data poisoning. [arXiv:1712.05526](#)
- Wang, B., et al. (2019). Neural Cleanse: Identifying and mitigating backdoor attacks in neural networks. *IEEE S&P 2019*. [DOI:10.1109/SP.2019.00031](#)
- Liu, K., Dolan-Gavitt, B., & Garg, S. (2018). Fine-pruning: Defending against backdooring attacks on deep neural networks. *RAID 2018*. [arXiv:1805.12185](#)